JavaScript Day 2 - Revision Notes

Table of Contents

Identifier Rules

An **identifier** is a unique name assigned to variables, functions, or objects in JavaScript. Understanding the rules for valid identifiers is crucial for writing error-free code.

What is an Identifier?

Think of identifiers as labels or names you give to different parts of your code. Just like how you have a name, every variable, function, or object in your code needs a unique identifier.

The Five Essential Rules

1. Characters Allowed

Identifiers can contain:

- **Letters**: Both uppercase (A-Z) and lowercase (a-z)
- **Digits**: Numbers 0-9

- **Underscores**: _
- **Dollar signs**: $

```javascript
// Valid examples
let userName123;
let _privateVariable;
let $specialValue;
let total_AMOUNT;
```

Important: No other special characters are allowed. Characters like @, #, %, &, -, +, etc., will cause errors.

2. No Spaces

Identifiers cannot contain spaces. This is a hard rule in JavaScript.

```javascript
// Invalid
let user name;     // ✗ Error
let total cost;    // ✗ Error

// Valid alternatives
let userName;      // ✓ Use camelCase
let user_name;     // ✓ Use underscores
let totalCost;     // ✓ Use camelCase
```

3. Starting Character

Identifiers **must** begin with:

- A letter (a-z, A-Z)
- An underscore (_)
- A dollar sign ($)

Identifiers **cannot** start with a digit (0-9).

```javascript
```

```
// Valid
let age;          // ✓ Starts with letter
let _private;     // ✓ Starts with underscore
let $price;       // ✓ Starts with dollar sign

// Invalid
let 123name;      // ✗ Cannot start with digit
let 9lives;       // ✗ Cannot start with digit
```

Why this rule? Starting with a digit would make it ambiguous whether you're writing a number or a variable name.

4. Case Sensitivity

JavaScript is **case-sensitive**, meaning it treats uppercase and lowercase letters as completely different.

javascript

```
let userAge = 25;
let userage = 30;
let UserAge = 35;
let USERAGE = 40;

// These are FOUR DIFFERENT variables!
console.log(userAge);  // 25
console.log(userage);  // 30
console.log(UserAge);  // 35
console.log(USERAGE);  // 40
```

Pro Tip: Be consistent with your casing to avoid confusion. If you name a variable `userName`, don't refer to it later as `username` or `UserName`.

5. Reserved Words

You cannot use JavaScript **keywords** or **reserved words** as identifiers. These words have special meanings in JavaScript.

Common reserved words include:

- `let`, `const`, `var` (variable declarations)
- `if`, `else`, `while`, `for` (control structures)
- `function`, `return` (function-related)
- `true`, `false`, `null`, `undefined` (special values)
- `class`, `extends`, `import`, `export` (advanced features)

```javascript
// Invalid - using reserved words
let let = 10;        // ❌ Error
let if = 20;         // ❌ Error
let function = 30;   // ❌ Error

// Valid alternatives
let letVariable = 10;  // ✓
let ifCondition = 20;  // ✓
let functionName = 30; // ✓
```

Complete list: There are about 60+ reserved words in JavaScript. You don't need to memorize them all - your code editor will usually highlight them, and you'll get an error if you try to use one.

Valid vs Invalid Identifiers

Let's look at concrete examples to solidify your understanding.

✓ Valid Identifiers

Identifier	Reason
`price123`	Contains letters and digits ✓
`total_cost`	Uses underscore ✓
`$amount`	Starts with dollar sign ✓
`_private`	Starts with underscore ✓
`userName`	Uses camelCase (standard convention) ✓
`MAX_VALUE`	All caps with underscores (constant convention) ✓
`item1`	Letter followed by digit ✓
`$_`	Just special characters (unusual but valid) ✓

✗ Invalid Identifiers

Identifier	Reason
`123price`	Cannot start with a digit ❌
`old price`	Contains a space ❌
`total-cost`	Hyphen is not allowed (interpreted as subtraction) ❌
`let`	Reserved keyword ❌
`user@name`	@ symbol not allowed ❌
`total#`	# symbol not allowed ❌
`price%`	% symbol not allowed ❌
`my-var`	Hyphen interpreted as minus operator ❌

Common Mistakes to Avoid

1. Hyphens vs Underscores

```javascript
// Invalid
let user-name = "Alice";  // ❌ JavaScript thinks: user minus name

// Valid
let user_name = "Alice";  // ✓ Underscore is allowed
let userName = "Alice";   // ✓ CamelCase (preferred)
```

2. Starting with Numbers

```javascript
// Invalid
let 1stPlace = "Gold";   // ❌ Cannot start with digit
let 99balloons = 99;     // ❌ Cannot start with digit

// Valid
let firstPlace = "Gold"; // ✓ Spell out the number
let place1st = "Gold";   // ✓ Number at end is fine
let balloons99 = 99;     // ✓ Number at end is fine
```

3. Special Characters

```javascript
// Invalid
let my$variable = 10;   // ❌ Wait, this is actually VALID!
let my#variable = 10;   // ❌ # is not allowed

// Remember: Only _ and $ are allowed as special characters
let my_variable = 10;   // ✓
let my$variable = 10;   // ✓
```

Naming Conventions

While JavaScript allows many valid identifier patterns, following **naming conventions** makes your code more readable and maintainable. These conventions are not enforced by JavaScript, but they are widely adopted by the programming community.

1. camelCase (Recommended ⭐)

Rule: First word lowercase, subsequent words capitalized.

Used for: Variables, functions, and method names

```javascript
let userName = "Alice";
let totalPrice = 99.99;
let isLoggedIn = true;
let calculateTotalCost = function() { };
let getUserInfo = function() { };
```

Why camelCase?

- It's the JavaScript standard
- Easy to read: `getUserEmailAddress` vs `getuseremailaddress`
- Consistent with most JavaScript libraries and frameworks

Examples from real code:

```javascript
```

```javascript
// Variables
let firstName = "John";
let lastName = "Doe";
let dateOfBirth = "1990-01-01";
let emailAddress = "john@example.com";

// Functions
function calculateAge(birthYear) {
    return 2024 - birthYear;
}

function sendEmailNotification(email, message) {
    // code here
}
```

2. snake_case

Rule: Words separated by underscores, all lowercase.

Used for: Some developers prefer this, especially those from Python background

```javascript
let user_name = "Alice";
let total_price = 99.99;
let is_logged_in = true;
let calculate_total = function() { };
```

Note: While valid, this is **not** the JavaScript convention. However, you might see it in:

- Database field names
- API responses
- Configuration files

3. PascalCase

Rule: Every word starts with a capital letter (including the first).

Used for: Classes, constructors, and components (React)

```javascript
```

```
// Classes and Constructors
class UserAccount {
    constructor(name) {
        this.name = name;
    }
}

class ShoppingCart {
    // class code
}

// React Components
function UserProfile() {
    return <div>Profile</div>;
}
```

When to use PascalCase:

- Creating a class: `class BankAccount { }`
- Creating a constructor function: `function Person(name) { }`
- React components: `function HomePage() { }`

4. SCREAMING_SNAKE_CASE

Rule: All uppercase letters with underscores.

Used for: Constants (values that never change)

```
javascript

const MAX_LOGIN_ATTEMPTS = 3;
const API_KEY = "abc123xyz";
const PI = 3.14159;
const DATABASE_URL = "mongodb://localhost:27017";
```

Why for constants?

- Makes it immediately obvious that this value shouldn't be changed
- Easy to spot constants in your code

5. Hungarian Notation (Rarely Used)

Some developers prefix variables with type indicators:

```javascript
let strName = "Alice";    // str = string
let numAge = 25;          // num = number
let bIsActive = true;     // b = boolean
```

Note: This is **not recommended** in modern JavaScript. TypeScript provides better type safety.

Best Practices Summary

Use Case	Convention	Example
Variables	camelCase	`userName`, `totalPrice`
Functions	camelCase	`calculateTotal`, `getUserData`
Constants	SCREAMING_SNAKE_CASE	`MAX_SIZE`, `API_URL`
Classes	PascalCase	`UserAccount`, `ShoppingCart`
Private variables	_camelCase	`_privateVar` (convention, not enforced)

Descriptive Names

Beyond conventions, use **descriptive names**:

```javascript
```

```javascript
// Bad - unclear names
let x = 25;
let y = "John";
let z = true;

// Good - descriptive names
let userAge = 25;
let firstName = "John";
let isEmailVerified = true;

// Bad - too short
let fn = "Alice";
let ln = "Smith";

// Good - clear and readable
let firstName = "Alice";
let lastName = "Smith";

// Bad - abbreviations
let usrAddr = "123 Main St";
let empSal = 50000;

// Good - spelled out
let userAddress = "123 Main St";
let employeeSalary = 50000;
```

Booleans and Dynamic Typing

Booleans

A **Boolean** is a data type that represents one of two values: `true` or `false`. Booleans are fundamental to programming logic and decision-making.

Basic Boolean Usage

javascript

```javascript
let isLoggedIn = true;
let isOver18 = false;
let hasPermission = true;
let isEmailVerified = false;
```

Boolean in Conditions

Booleans are typically used in conditional statements:

```javascript
let isStudent = true;

if (isStudent) {
    console.log("Student discount applied!");
} else {
    console.log("Regular price.");
}
```

Boolean Expressions

You can create boolean values through comparisons:

```javascript
let age = 20;
let isAdult = age >= 18;        // true
console.log(isAdult);           // true

let score = 75;
let isPassing = score >= 60;    // true
let isFailing = score < 60;     // false

let temperature = 30;
let isHot = temperature > 25;   // true
let isCold = temperature < 10;  // false
```

Common Boolean Use Cases

```javascript
```

```
// Toggle states
let isMenuOpen = false;
let isDarkMode = true;
let isPlaying = false;

// User states
let isLoggedIn = true;
let isAdmin = false;
let isVerified = true;

// Feature flags
let isFeatureEnabled = true;
let isBetaUser = false;

// Validation
let isEmailValid = true;
let isPasswordStrong = false;
let isFormComplete = true;
```

Truthy and Falsy Values (Advanced)

In JavaScript, values can be "truthy" or "falsy" even if they're not actual booleans:

Falsy values (evaluate to `false`):

```javascript
false
0
"" (empty string)
null
undefined
NaN
```

Everything else is truthy, including:

```javascript
true
1 (or any non-zero number)
"hello" (any non-empty string)
[] (arrays)
{} (objects)
```

Example:

```javascript
if (0) {
    console.log("This won't run");  // 0 is falsy
}

if (1) {
    console.log("This will run");   // 1 is truthy
}

if ("hello") {
    console.log("This will run");   // non-empty string is truthy
}
```

Dynamic Typing: JavaScript vs TypeScript

Understanding the difference between dynamically typed and statically typed languages is important.

JavaScript: Dynamically Typed

In JavaScript, a variable's **type can change** during execution. You can assign different types of values to the same variable.

```javascript
let x = 5;              // x is a number
console.log(typeof x);  // "number"

x = "hello";            // Now x is a string - ALLOWED!
console.log(typeof x);  // "string"

x = true;               // Now x is a boolean - ALLOWED!
console.log(typeof x);  // "boolean"

x = [1, 2, 3];          // Now x is an array - ALLOWED!
console.log(typeof x);  // "object"
```

Advantages:

- Flexible and quick to write
- Easy for beginners
- Less verbose code

Disadvantages:

- Can lead to unexpected bugs
- Harder to catch errors during development
- Type-related errors only appear at runtime

Example of a potential problem:

```javascript
function addNumbers(a, b) {
  return a + b;
}

console.log(addNumbers(5, 10));     // 15 ✓
console.log(addNumbers("5", 10));   // "510" ⚠ Unexpected string concatenation!
console.log(addNumbers(5, "10"));   // "510" ⚠
```

TypeScript: Statically Typed

TypeScript is a superset of JavaScript developed by Microsoft. It adds **static typing**, meaning you declare variable types and they cannot change.

```typescript
let x: number = 5;      // x must be a number
console.log(x);         // 5

x = "hello";          // ✗ ERROR: Type 'string' is not assignable to type 'number'
x = true;             // ✗ ERROR: Type 'boolean' is not assignable to type 'number'

// This will only accept numbers
x = 10;               // ✓ This works
```

Advantages:

- Catches errors during development (before running)
- Better IDE support (autocomplete, suggestions)
- Self-documenting code
- Easier to maintain large codebases

Disadvantages:

- More verbose
- Learning curve

- Requires compilation step

TypeScript example with function:

```typescript
function addNumbers(a: number, b: number): number {
  return a + b;
}

console.log(addNumbers(5, 10));     // 15 ✓
console.log(addNumbers("5", 10));   // ✗ ERROR at compile time!
console.log(addNumbers(5, "10"));   // ✗ ERROR at compile time!
```

When to Use Each

Use JavaScript when:

- Learning programming
- Building small projects or prototypes
- Need maximum flexibility
- Working with a team comfortable with dynamic typing

Use TypeScript when:

- Building large applications
- Working in a team (types serve as documentation)
- Want to catch errors early
- Need better refactoring support

Practical Comparison

```javascript
// JavaScript - Flexible but potentially problematic
let userAge = 25;
userAge = "twenty-five";  // Allowed but probably not intended
userAge = null;           // Allowed but might break calculations

function calculateDiscount(age) {
  return age * 0.1;  // What if age is a string?
}
```

```typescript
// TypeScript - More rigid but safer
let userAge: number = 25;
userAge = "twenty-five";  // ❌ Compile error - caught immediately!
userAge = null;           // ❌ Compile error

function calculateDiscount(age: number): number {
    return age * 0.1;  // Guaranteed age is a number
}
```

Strings in JavaScript

A **string** is a sequence of characters used to represent text. Strings are one of the most commonly used data types in programming.

String Declaration

JavaScript provides three ways to create strings:

1. Single Quotes (`'...'`)

```javascript
let firstName = 'Alice';
let greeting = 'Hello, World!';
let message = 'It\'s a beautiful day';  // Escape quote with \
```

2. Double Quotes (`"..."`)

```javascript
let firstName = "Alice";
let greeting = "Hello, World!";
let message = "She said, \"Hello!\"";  // Escape quote with \
```

3. Backticks / Template Literals (`` `...` ``)

```javascript
let firstName = `Alice`;
let greeting = `Hello, World!`;
let message = `It's a "beautiful" day`;  // No escaping needed!
```

Which should you use?

- Single and double quotes are functionally identical
- Choose one and be consistent (many developers prefer single quotes)
- Use backticks for template literals (when you need to embed variables or expressions)

Nesting Quotes

When you need to include a quote character inside your string, you have several options:

Option 1: Use Different Quote Types

```javascript
// Use double quotes on outside, single quotes inside
let message1 = "It's a beautiful day";          // ✓ Works!
let message2 = "He said, 'Hello there!'";       // ✓ Works!

// Use single quotes on outside, double quotes inside
let message3 = 'She said, "Good morning!"';     // ✓ Works!
let message4 = 'The word "hello" is friendly';  // ✓ Works!
```

Option 2: Escape Characters with Backslash

```javascript
// Escape single quote inside single quotes
let message1 = 'It\'s a beautiful day';         // ✓ Works!

// Escape double quote inside double quotes
let message2 = "He said, \"Hello there!\"";     // ✓ Works!
```

Option 3: Use Template Literals (Backticks)

```javascript
// No escaping needed!
let message1 = `It's a beautiful day`;          // ✓ Works!
let message2 = `She said, "Good morning!"`;     // ✓ Works!
let message3 = `Use 'any' "quotes" you want!`;  // ✓ Works!
```

Common Mistakes with Quotes

❌ **Wrong: Same quotes without escaping**

```javascript
let message = 'It's a day';  // ✘ ERROR!
// JavaScript thinks the string ends at the second quote
// It sees: 'It' followed by confusing text: s a day';
```

✓ Right: Solutions

```javascript
let message1 = "It's a day";     // ✓ Different outer quotes
let message2 = 'It\'s a day';    // ✓ Escaped quote
let message3 = `It's a day`;     // ✓ Template literal
```

String Properties and Methods

Length Property

```javascript
let name = "Alice";
console.log(name.length);  // 5

let message = "Hello, World!";
console.log(message.length);  // 13

let empty = "";
console.log(empty.length);  // 0
```

Common String Methods (Preview)

```javascript
```

```javascript
let text = "Hello, World!";

// Convert to uppercase
console.log(text.toUpperCase());  // "HELLO, WORLD!"

// Convert to lowercase
console.log(text.toLowerCase());  // "hello, world!"

// Extract part of string
console.log(text.slice(0, 5));    // "Hello"

// Replace text
console.log(text.replace("World", "JavaScript"));  // "Hello, JavaScript!"
```

Note: We'll cover string methods in detail in future lessons.

Escape Sequences

Special characters that start with a backslash (`\`):

javascript

```javascript
let newLine = "First line\nSecond line";
console.log(newLine);
// Output:
// First line
// Second line

let tab = "Name:\tAlice";
console.log(tab);
// Output: Name:    Alice

let backslash = "This is a backslash: \\";
console.log(backslash);
// Output: This is a backslash: \

let quote = "She said, \"Hi!\"";
console.log(quote);
// Output: She said, "Hi!"
```

Common escape sequences:

- `\n` - New line
- `\t` - Tab
- `\\` - Backslash

- `\'` - Single quote
- `\"` - Double quote
- `\r` - Carriage return

String Indexing

JavaScript uses **0-based indexing** for strings. This means the first character is at position 0, not position 1.

Understanding Indexing

Think of a string as a row of boxes, each containing one character. Each box has a number (index) starting from 0.

```
String: "London"

Index:   0   1   2   3   4   5
Value:  'L' 'o' 'n' 'd' 'o' 'n'
```

Accessing Characters

Use square brackets `[]` with the index number:

javascript

```javascript
let city = "London";

console.log(city[0]);  // "L" - first character
console.log(city[1]);  // "o" - second character
console.log(city[2]);  // "n" - third character
console.log(city[3]);  // "d" - fourth character
console.log(city[4]);  // "o" - fifth character
console.log(city[5]);  // "n" - sixth (last) character
```

First and Last Characters

javascript

```javascript
```

```javascript
let word = "JavaScript";

// First character
let first = word[0];
console.log(first);  // "J"

// Last character using length
let last = word[word.length - 1];
console.log(last);  // "t"

// Why length - 1?
// Because length is 10, but last index is 9 (0-based indexing)
console.log(word.length);      // 10
console.log(word[9]);          // "t"
console.log(word[10]);         // undefined (out of bounds)
```

Out of Bounds Access

Accessing an index that doesn't exist returns `undefined`:

javascript

```javascript
let name = "Alice";  // length is 5, valid indices: 0-4

console.log(name[0]);   // "A" ✓
console.log(name[4]);   // "e" ✓
console.log(name[5]);   // undefined (no character at index 5)
console.log(name[10]);  // undefined
console.log(name[-1]);  // undefined (negative indices don't work like Python)
```

Practical Examples

Example 1: Checking First Character

javascript

```javascript
let email = "alice@example.com";

if (email[0] === '@') {
    console.log("Email cannot start with @");
} else {
    console.log("Email format might be valid");
}
```

Example 2: Getting Initials

```javascript
let firstName = "Alice";
let lastName = "Smith";

let initials = firstName[0] + lastName[0];
console.log(initials);  // "AS"

// With dots
let formalInitials = firstName[0] + "." + lastName[0] + ".";
console.log(formalInitials);  // "A.S."
```

Example 3: Character Validation

```javascript
let password = "Pass123";

let firstChar = password[0];
let lastChar = password[password.length - 1];

console.log("First character:", firstChar);  // "P"
console.log("Last character:", lastChar);    // "3"

if (lastChar >= '0' && lastChar <= '9') {
   console.log("Password ends with a number");
}
```

Important Notes

1. **Strings are Immutable**: You cannot change individual characters

```javascript
let word = "Hello";
word[0] = "J";  // This does NOT work!
console.log(word);  // Still "Hello"

// To "change" a string, create a new one
let newWord = "J" + word.slice(1);
console.log(newWord);  // "Jello"
```

2. **Length vs Last Index**

```javascript
let text = "Hello";
console.log(text.length);       // 5
console.log(text[text.length]); // undefined (index 5 doesn't exist)
console.log(text[text.length - 1]); // "o" (correct last character)
```

3. **Spaces Count Too**

```javascript
let text = "Hello World";
console.log(text.length); // 11 (space counts!)
console.log(text[5]);     // " " (space character)
```

String Concatenation

Concatenation means joining strings together. JavaScript provides the `+` operator for this purpose.

Basic Concatenation

```javascript
// Joining two strings
let firstName = "Alice";
let lastName = "Smith";
let fullName = firstName + lastName;
console.log(fullName); // "AliceSmith"

// Adding space
let fullNameWithSpace = firstName + " " + lastName;
console.log(fullNameWithSpace); // "Alice Smith"
```

Multiple Concatenations

```javascript
let greeting = "Hello";
let name = "World";
let punctuation = "!";

let message = greeting + ", " + name + punctuation;
console.log(message); // "Hello, World!"
```

Concatenating Numbers and Strings

When you use (+) with a string and a number, JavaScript converts the number to a string:

```javascript
// Number + String = String
let score = 100;
let message1 = "Your score is: " + score;
console.log(message1);  // "Your score is: 100"
console.log(typeof message1);  // "string"

// String + Number = String
let message2 = "Player " + 1;
console.log(message2);  // "Player 1"

// Multiple numbers and strings
let age = 25;
let message3 = "I am " + age + " years old";
console.log(message3);  // "I am 25 years old"
```

Concatenation Examples

Example 1: Building Messages

```javascript
let userName = "Bob";
let points = 150;

let welcomeMsg = "Welcome back, " + userName + "!";
let scoreMsg = "You have " + points + " points.";

console.log(welcomeMsg);  // "Welcome back, Bob!"
console.log(scoreMsg);    // "You have 150 points."
```

Example 2: Creating URLs

```javascript
```

```javascript
let domain = "example.com";
let protocol = "https://";
let page = "/about";

let fullURL = protocol + domain + page;
console.log(fullURL);  // "https://example.com/about"
```

Example 3: Building File Names

```javascript
let fileName = "report";
let fileType = ".pdf";
let year = 2024;

let fullFileName = fileName + "_" + year + fileType;
console.log(fullFileName);  // "report_2024.pdf"
```

The += Operator for Concatenation

You can use `+=` to append to an existing string:

```javascript
let message = "Hello";
message += " ";       // message = message + " "
message += "World";   // message = message + "World"
message += "!";       // message = message + "!"

console.log(message);  // "Hello World!"
```

Practical example:

```javascript
let htmlContent = "<div>";
htmlContent += "<h1>Title</h1>";
htmlContent += "<p>Paragraph</p>";
htmlContent += "</div>";

console.log(htmlContent);
// Output: <div><h1>Title</h1><p>Paragraph</p></div>
```

Important: Number Addition vs String Concatenation

Be careful when mixing `+` operator with numbers and strings:

```javascript
// All numbers - addition
console.log(10 + 20);          // 30 (number addition)

// String first - concatenation
console.log("10" + 20);        // "1020" (string concatenation)

// Number first, then string - concatenation
console.log(10 + "20");        // "1020" (string concatenation)

// Mixed operations
console.log(10 + 20 + "30");   // "3030" (10+20=30, then "30"+"30")
console.log("10" + 20 + 30);   // "102030" (all concatenation)

// Using parentheses
console.log("Result: " + (10 + 20));  // "Result: 30" (forces addition first)
```

Rule: If **any** operand in a `+` operation is a string, JavaScript performs concatenation (converts everything to strings).

Concatenation vs Template Literals

Concatenation can get messy with complex strings:

```javascript
// Concatenation (hard to read)
let name = "Alice";
let age = 25;
let city = "New York";
let message = "My name is " + name + ", I am " + age + " years old, and I live in " + city + ".";

// Template Literal (easier to read) - We'll cover this next!
let betterMessage = `My name is ${name}, I am ${age} years old, and I live in ${city}.`;
```

Null vs Undefined

Both `null` and `undefined` represent the absence of a value, but they're used in different contexts.

Undefined

Undefined means a variable has been declared but not assigned a value yet.

When You Get Undefined

1. **Variable declared but not initialized:**

```javascript
let score;
console.log(score);       // undefined
console.log(typeof score); // "undefined"
```

2. **Function with no return value:**

```javascript
function greet() {
  console.log("Hello!");
  // No return statement
}

let result = greet();
console.log(result);  // undefined
```

3. **Accessing non-existent object property:**

```javascript
let user = { name: "Alice" };
console.log(user.age);  // undefined (property doesn't exist)
```

4. **Array element that doesn't exist:**

```javascript
let colors = ["red", "blue"];
console.log(colors[5]);  // undefined
```

5. **Function parameter not provided:**

```javascript
```

```
function greet(name) {
    console.log(name);
}
greet();  // undefined (no argument passed)
```

Example Usage

```
javascript

let userInput;  // Declared but not assigned

if (userInput === undefined) {
    console.log("Please provide input");
} else {
    console.log("Input received:", userInput);
}
```

Null

Null represents the **intentional absence** of a value. You explicitly assign `null` to indicate "no value" or "empty."

When to Use Null

1. **Intentionally setting something as empty:**

```
javascript

let currentUser = null;  // No user logged in
console.log(currentUser);       // null
console.log(typeof currentUser); // "object" (this is a JavaScript quirk!)
```

2. **Resetting a value:**

```
javascript

let winner = "Alice";
console.log(winner);  // "Alice"

// Game resets
winner = null;
console.log(winner);  // null (intentionally cleared)
```

3. **API responses:**

```javascript
let userData = {
    name: "Bob",
    email: "bob@example.com",
    phone: null  // User hasn't provided phone number
};
```

Key Differences

Aspect	Undefined	Null
Meaning	"Not assigned yet"	"Intentionally empty"
Assignment	Automatic (by JavaScript)	Manual (by programmer)
typeof	"undefined"	"object" (quirk!)
Use Case	Default for uninitialized variables	Explicitly clearing/resetting values

Practical Comparison

```javascript
// Undefined - forgot to assign
let firstName;
console.log(firstName);       // undefined
console.log(typeof firstName); // "undefined"

// Null - intentionally empty
let middleName = null;  // Person has no middle name
console.log(middleName);       // null
console.log(typeof middleName); // "object" (historical bug in JavaScript)
```

Checking for Null or Undefined

```javascript
```

```javascript
let value1;         // undefined
let value2 = null;  // null

// Check for undefined
if (value1 === undefined) {
    console.log("value1 is undefined");
}

// Check for null
if (value2 === null) {
    console.log("value2 is null");
}

// Check for either (common pattern)
if (value1 == null) {  // Note: using == (loose equality)
    console.log("value1 is null or undefined");
}
// This works because: undefined == null is true
```

The typeof Quirk

```javascript
console.log(typeof undefined);  // "undefined" ✓ Makes sense
console.log(typeof null);       // "object" ⚠️ Historical bug!

// To properly check for null, use strict equality:
let value = null;
if (value === null) {
    console.log("It's null");
}

// Don't rely on typeof for null
if (typeof value === "object") {
    // This could be null OR an actual object!
    console.log("Could be null or an object");
}
```

Real-World Examples

Example 1: User Login State

```javascript
```

```javascript
let loggedInUser = null;  // No one logged in initially

function login(username) {
    loggedInUser = username;
}

function logout() {
    loggedInUser = null;  // Explicitly clear the user
}

login("Alice");
console.log(loggedInUser);  // "Alice"

logout();
console.log(loggedInUser);  // null
```

Example 2: Optional Form Fields

```javascript
let formData = {
    firstName: "John",
    lastName: "Doe",
    middleName: null,    // Optional field - intentionally null
    nickname: undefined   // Not provided at all
};
```

Example 3: Database Records

```javascript
let product = {
    id: 101,
    name: "Laptop",
    discount: null,      // No discount currently
    rating: undefined     // No ratings yet
};
```

Best Practices

1. **Use undefined for "not yet initialized"**

```javascript
```

```
let userChoice;  // Will be set later
```

2. **Use null for "intentionally empty"**

```
javascript

let selectedItem = null;  // User hasn't selected anything
```

3. **Always use strict equality (===) to check**

```
javascript

if (value === null) { }      // Check for null
if (value === undefined) { } // Check for undefined
```

4. **Check for both with loose equality if needed**

```
javascript

if (value == null) {  // true for both null AND undefined
    console.log("No value");
}
```

Output Methods

console.log()

The `console.log()` method is used to print output to the browser's console. It's essential for debugging and testing.

Basic Usage

```
javascript

console.log("Hello, World!");
```

Multiple Arguments

You can pass multiple arguments separated by commas:

```
javascript
```

```javascript
console.log("Name:", "Alice");
console.log("Age:", 25);
console.log("City:", "New York");

// Output:
// Name: Alice
// Age: 25
// City: New York
```

Printing Variables

```javascript
let name = "Bob";
let age = 30;
let isStudent = false;

console.log(name);     // Bob
console.log(age);      // 30
console.log(isStudent);// false
```

Combining Text and Variables

```javascript
let product = "Laptop";
let price = 999;

console.log("Product:", product);
console.log("Price:", price);

// Or combine in one log
console.log("Product:", product, "- Price:", price);
// Output: Product: Laptop - Price: 999
```

Printing Calculations

```javascript
console.log("Sum:", 10 + 20);       // Sum: 30
console.log("Result:", 50 / 2);     // Result: 25
console.log("5 * 5 =", 5 * 5);      // 5 * 5 = 25
```

Debugging with console.log()

```javascript
let x = 10;
console.log("Before:", x);  // Before: 10

x = x + 5;
console.log("After:", x);   // After: 15

x *= 2;
console.log("Final:", x);   // Final: 30
```

Other Console Methods (Preview)

While `console.log()` is the most common, there are other useful console methods:

```javascript
// Warning message (yellow in console)
console.warn("This is a warning!");

// Error message (red in console)
console.error("This is an error!");

// Info message
console.info("This is information");

// Clear the console
console.clear();
```

Template Literals

Template literals (also called template strings) provide a more elegant way to work with strings. They use **backticks** (`` ` ``) instead of quotes and allow you to embed expressions directly inside strings.

Basic Syntax

Instead of quotes, use backticks:

```javascript
```

```javascript
// Regular string
let message1 = "Hello, World!";

// Template literal
let message2 = `Hello, World!`;

// Both produce the same output
console.log(message1);  // Hello, World!
console.log(message2);  // Hello, World!
```

Embedding Variables with `${}`

Use `${}` to insert variables or expressions into a string:

javascript

```javascript
let name = "Alice";
let age = 25;

// Old way (concatenation)
let message1 = "My name is " + name + " and I am " + age + " years old.";

// New way (template literal)
let message2 = `My name is ${name} and I am ${age} years old.`;

console.log(message1);  // My name is Alice and I am 25 years old.
console.log(message2);  // My name is Alice and I am 25 years old.
```

Why Template Literals Are Better

1. Cleaner Syntax

javascript

```javascript
// Concatenation (messy)
let product = "Laptop";
let price = 999;
let message = "The " + product + " costs $" + price + " dollars.";

// Template literal (clean)
let betterMessage = `The ${product} costs $${price} dollars.`;
```

2. Easy Multi-line Strings

javascript

```javascript
// Regular strings (need escape characters)
let oldWay = "Line 1\nLine 2\nLine 3";

// Template literals (natural multi-line)
let newWay = `Line 1
Line 2
Line 3`;

console.log(newWay);
// Output:
// Line 1
// Line 2
// Line 3
```

3. Embed Expressions

You can put any JavaScript expression inside `${}`:

```javascript
let a = 10;
let b = 20;

console.log(`Sum: ${a + b}`);          // Sum: 30
console.log(`Product: ${a * b}`);      // Product: 200
console.log(`${a} + ${b} = ${a + b}`); // 10 + 20 = 30
```

Practical Examples

Example 1: User Greeting

```javascript
let userName = "Bob";
let hour = 14;

let greeting = `Good ${hour < 12 ? 'morning' : 'afternoon'}, ${userName}!`;
console.log(greeting); // Good afternoon, Bob!
```

Example 2: Shopping Cart

```javascript
```

```javascript
let item = "Apples";
let quantity = 5;
let price = 2.5;
let total = quantity * price;

let receipt = `
Item: ${item}
Quantity: ${quantity}
Price per unit: $${price}
Total: $${total}
`;

console.log(receipt);
// Output:
// Item: Apples
// Quantity: 5
// Price per unit: $2.5
// Total: $12.5
```

Example 3: HTML Generation

```javascript
let title = "Welcome";
let content = "Hello, World!";

let html = `
<div class="card">
  <h1>${title}</h1>
  <p>${content}</p>
</div>
`;

console.log(html);
```

Example 4: Dynamic Messages

```javascript
```

```javascript
let playerName = "Alice";
let score = 1500;
let level = 5;

let statusMessage = `Player ${playerName} is on level ${level} with ${score} points!`;
console.log(statusMessage);
// Output: Player Alice is on level 5 with 1500 points!
```

Advanced Features

Expressions and Function Calls

javascript

```javascript
function getFullName(first, last) {
   return `${first} ${last}`;
}

let firstName = "John";
let lastName = "Doe";
let age = 30;

let message = `${getFullName(firstName, lastName)} is ${age} years old.`;
console.log(message);  // John Doe is 30 years old.
```

Conditional Logic

javascript

```javascript
let temperature = 30;
let weather = `It's ${temperature > 25 ? 'hot' : 'cool'} today!`;
console.log(weather);  // It's hot today!

let age = 20;
let status = `You are ${age >= 18 ? 'an adult' : 'a minor'}.`;
console.log(status);  // You are an adult.
```

Comparison: Concatenation vs Template Literals

javascript

```javascript
let name = "Alice";
let age = 25;
let city = "New York";

// Concatenation (hard to read and write)
let msg1 = "Hi, I'm " + name + ". I'm " + age + " years old and I live in " + city + ".";

// Template Literal (easy to read and write)
let msg2 = `Hi, I'm ${name}. I'm ${age} years old and I live in ${city}.`;

console.log(msg1);  // Hi, I'm Alice. I'm 25 years old and I live in New York.
console.log(msg2);  // Hi, I'm Alice. I'm 25 years old and I live in New York.
```

Best Practice

Use template literals whenever you need to:

- Embed variables in strings
- Write multi-line strings
- Include expressions or calculations
- Build HTML/XML strings

Use regular strings for:

- Simple, static strings with no variables
- When using older JavaScript environments (ES5)

Comparison Operators

Comparison operators are used to compare two values and return a boolean result (`true` or `false`). They're essential for making decisions in your code.

The Six Main Comparison Operators

1. Greater Than (`>`)

Checks if the left value is greater than the right value.

```javascript
```

```
console.log(10 > 5);   // true
console.log(5 > 10);   // false
console.log(5 > 5);    // false (not greater, just equal)

let age = 20;
console.log(age > 18);  // true
```

2. Less Than (`<`)

Checks if the left value is less than the right value.

javascript

```
console.log(5 < 10);   // true
console.log(10 < 5);   // false
console.log(5 < 5);    // false

let score = 45;
console.log(score < 50);  // true
```

3. Greater Than or Equal To (`>=`)

Checks if the left value is greater than OR equal to the right value.

javascript

```
console.log(10 >= 5);   // true (10 is greater)
console.log(5 >= 10);   // false
console.log(5 >= 5);    // true (equal counts!)

let age = 18;
console.log(age >= 18);  // true (can vote)
```

4. Less Than or Equal To (`<=`)

Checks if the left value is less than OR equal to the right value.

javascript

```
console.log(5 <= 10);   // true
console.log(10 <= 5);   // false
console.log(5 <= 5);    // true

let temperature = 25;
console.log(temperature <= 30);  // true
```

5. Not Equal To (`!=`)

Checks if two values are not equal (loose comparison - converts types).

```javascript
console.log(5 != 4);    // true
console.log(5 != 5);    // false
console.log(5 != "5");  // false (converts string to number)

let userInput = 10;
console.log(userInput != 0);  // true
```

6. Loose Equality (`==`)

Checks if two values are equal (converts types if needed).

```javascript
console.log(5 == 5);     // true
console.log(5 == "5");   // true (string converted to number)
console.log(1 == true);  // true (boolean converted to number)
console.log(0 == false); // true

let x = 10;
let y = "10";
console.log(x == y);  // true (type conversion happens)
```

Practical Examples

Example 1: Age Verification

```javascript
let age = 17;

if (age >= 18) {
   console.log("You can vote");
} else {
   console.log("You cannot vote yet");
}
// Output: You cannot vote yet
```

Example 2: Temperature Check

```javascript
```

```
let temperature = 30;

if (temperature > 25) {
   console.log("It's hot outside");
} else {
   console.log("It's cool outside");
}
// Output: It's hot outside
```

Example 3: Password Length

javascript

```
let password = "abc123";
let minLength = 8;

if (password.length >= minLength) {
   console.log("Password is strong enough");
} else {
   console.log("Password is too short");
}
// Output: Password is too short
```

Example 4: Grade Checking

javascript

```
let score = 85;

if (score >= 90) {
   console.log("Grade: A");
} else if (score >= 80) {
   console.log("Grade: B");
} else if (score >= 70) {
   console.log("Grade: C");
} else {
   console.log("Grade: F");
}
// Output: Grade: B
```

Comparison Results Summary

Comparison	Result	Explanation
`10 > 5`	`true`	10 is greater than 5

Comparison	Result	Explanation
`2 < 1`	`false`	2 is not less than 1
`5 >= 5`	`true`	5 equals 5
`3 <= 4`	`true`	3 is less than 4
`5 != 4`	`true`	5 is not equal to 4
`5 == "5"`	`true`	Loose equality (type conversion)

Loose vs Strict Equality

Understanding the difference between loose (`==`) and strict (`===`) equality is crucial for writing bug-free JavaScript code.

Loose Equality (`==`)

Loose equality compares values but **performs type conversion** if the types are different.

How It Works

javascript

```javascript
// Same types - straightforward comparison
console.log(5 == 5);      // true
console.log("hi" == "hi"); // true

// Different types - converts before comparing
console.log(5 == "5");    // true (string "5" converted to number 5)
console.log(1 == true);   // true (true converted to 1)
console.log(0 == false);  // true (false converted to 0)
console.log("" == false); // true (both are "falsy")
```

Examples of Type Conversion

javascript

```javascript
// Number vs String
console.log(10 == "10");      // true (string to number)
console.log(0 == "");         // true (empty string to 0)
console.log(0 == "0");        // true

// Boolean vs Number
console.log(true == 1);       // true
console.log(false == 0);      // true
console.log(true == 2);       // false (true is 1, not 2)

// Special cases
console.log(null == undefined); // true (special rule)
console.log(null == 0);       // false
console.log(undefined == 0);  // false
```

Problems with Loose Equality

Loose equality can lead to unexpected bugs:

```javascript
// Looks like it should be false, but...
console.log("" == 0);         // true ⚠️
console.log(" " == 0);        // true ⚠️
console.log("\n" == 0);       // true ⚠️

// Counter-intuitive
console.log(false == "false"); // false ⚠️
console.log(false == "0");    // true ⚠️
```

Strict Equality (`===`)

Strict equality compares both **value AND type** without any conversion. If the types are different, it immediately returns `false`.

How It Works

```javascript
```

```
// Same type and value - true
console.log(5 === 5);          // true
console.log("hi" === "hi");    // true
console.log(true === true);    // true

// Different types - always false
console.log(5 === "5");        // false (number vs string)
console.log(1 === true);       // false (number vs boolean)
console.log(0 === false);      // false (number vs boolean)
console.log("" === false);     // false (string vs boolean)
```

Clear and Predictable

javascript

```
// Number comparisons
console.log(10 === 10);        // true ✓
console.log(10 === "10");      // false ✓ (different types)

// String comparisons
console.log("hello" === "hello");  // true ✓
console.log("hello" === "Hello");  // false ✓ (case-sensitive)

// Boolean comparisons
console.log(true === true);    // true ✓
console.log(true === 1);       // false ✓ (different types)
console.log(false === 0);      // false ✓ (different types)

// Special values
console.log(null === undefined); // false ✓ (different types)
console.log(null === null);      // true ✓
console.log(undefined === undefined); // true ✓
```

Not Equal Operators

There are also "not equal" versions:

Loose Inequality (`!=`)

javascript

```
console.log(5 != 4);      // true
console.log(5 != "5");    // false (converts types)
console.log(1 != true);   // false (converts types)
```

Strict Inequality (`!==`)

```javascript
console.log(5 !== 4);     // true
console.log(5 !== "5");   // true (different types)
console.log(1 !== true);  // true (different types)
```

Side-by-Side Comparison

Expression	`==` (Loose)	`===` (Strict)
`5 == 5`	`true`	`true`
`5 == "5"`	`true` ⚠️	`false` ✓
`true == 1`	`true` ⚠️	`false` ✓
`0 == false`	`true` ⚠️	`false` ✓
`null == undefined`	`true` ⚠️	`false` ✓
`"" == 0`	`true` ⚠️	`false` ✓

Real-World Examples

Example 1: User Input Validation

```javascript
```

```javascript
let userInput = "10";  // From a form field (always a string)

// Loose equality - might hide bugs
if (userInput == 10) {
    console.log("Input is 10");  // This runs! (type conversion)
}

// Strict equality - more reliable
if (userInput === 10) {
    console.log("Input is 10");  // This does NOT run
}

// Proper way
let numericInput = Number(userInput);
if (numericInput === 10) {
    console.log("Input is 10");  // Correct!
}
```

Example 2: Boolean Checks

javascript

```javascript
let isActive = true;

// Loose equality - unreliable
if (isActive == 1) {
    console.log("Active");  // This runs (true converted to 1)
}

// Strict equality - clear intent
if (isActive === true) {
    console.log("Active");  // Better!
}

// Even better - direct boolean check
if (isActive) {
    console.log("Active");  // Best!
}
```

Example 3: API Response

javascript

```javascript
let apiResponse = {
  status: "200",  // String from API
  success: true
};

// Loose comparison
if (apiResponse.status == 200) {
  console.log("Success");  // Works but not ideal
}

// Strict comparison - catches type mismatches
if (apiResponse.status === 200) {
  console.log("Success");  // Doesn't run - reveals the bug!
}

// Fix: Convert to number first
if (Number(apiResponse.status) === 200) {
  console.log("Success");  // Correct approach
}
```

Best Practice: Always Use `===` and `!==`

Recommendation: Use strict equality (`===` and `!==`) by default. Only use loose equality (`==` and `!=`) when you specifically need type coercion and understand the implications.

```javascript
// ✓ GOOD - Explicit and predictable
if (age === 18) { }
if (name !== "") { }
if (score === 100) { }

// ✗ BAD - Unpredictable due to type coercion
if (age == 18) { }
if (name != "") { }
if (score == "100") { }
```

Exception: Checking for null or undefined

One common case where `==` is useful:

```javascript
```

```
let value = null;

// To check if value is either null or undefined
if (value == null) {  // true for both null and undefined
    console.log("No value");
}

// Equivalent strict equality (more verbose)
if (value === null || value === undefined) {
    console.log("No value");
}
```

Summary

Operator	Name	Behavior	Recommendation
`==`	Loose equality	Converts types	⚠️ Avoid unless needed
`===`	Strict equality	No conversion	✓ Use by default
`!=`	Loose inequality	Converts types	⚠️ Avoid unless needed
`!==`	Strict inequality	No conversion	✓ Use by default

Key Takeaway: Strict equality (`===`) is safer, more predictable, and helps catch bugs early. Make it your default choice!

String Comparison

JavaScript can compare strings using comparison operators, but it uses **Unicode (ASCII) values** rather than alphabetical order.

How String Comparison Works

Each character has a numeric Unicode value:

- Uppercase letters: A=65, B=66, ..., Z=90
- Lowercase letters: a=97, b=98, ..., z=122
- Digits: 0=48, 1=49, ..., 9=57

When comparing strings, JavaScript compares character by character using these values.

Basic Comparisons

```javascript
// Single character comparisons
console.log('a' > 'A');     // true (97 > 65)
console.log('b' < 'c');     // true (98 < 99)
console.log('A' < 'a');     // true (65 < 97)

// String comparison
console.log("apple" < "banana");  // true (a < b)
console.log("cat" > "bat");       // true (c > b)
```

Character-by-Character Comparison

JavaScript compares strings from left to right:

```javascript
// Compares first different character
console.log("apple" < "application");  // true
// Compares: a=a, p=p, p=p, l=l, e < i, so "apple" < "application"

console.log("hello" > "help");  // false
// Compares: h=h, e=e, l=l, l < p, so "hello" < "help"

console.log("cat" < "catalog");  // true
// Compares: c=c, a=a, t=t, then "cat" ends (shorter < longer when prefix matches)
```

Uppercase vs Lowercase

Important: Uppercase letters come BEFORE lowercase letters in ASCII/Unicode.

```javascript
// Uppercase < Lowercase
console.log('A' < 'a');     // true (65 < 97)
console.log('Z' < 'a');     // true (90 < 97)

// Comparisons with mixed case
console.log("Apple" < "apple");   // true (A < a)
console.log("HELLO" < "hello");   // true (H < h)

// This can be counter-intuitive!
console.log("Zoo" < "apple");     // true (Z < a)
```

Number Strings

When comparing number strings, remember they're compared as strings, not numbers:

```javascript
// String comparison (character by character)
console.log("10" < "2");   // true ⚠️
// Why? Compare first character: "1" < "2" (49 < 50)

console.log("100" < "20");  // true ⚠️
// Why? Compare first character: "1" < "2"

// Actual number comparison
console.log(10 < 2);       // false ✓
console.log(100 < 20);     // false ✓
```

To compare number strings as numbers:

```javascript
let str1 = "10";
let str2 = "2";

// Convert to numbers first
console.log(Number(str1) < Number(str2));  // false ✓

// Or use parseInt/parseFloat
console.log(parseInt(str1) < parseInt(str2));  // false ✓
```

Practical Examples

Example 1: Alphabetical Sorting

```javascript
let names = ["Charlie", "Alice", "Bob"];

// This won't work as expected with uppercase/lowercase mix
let name1 = "alice";
let name2 = "Bob";
console.log(name1 < name2);  // false (a > B in ASCII)

// Solution: Convert to same case before comparing
console.log(name1.toLowerCase() < name2.toLowerCase());  // true ✓
console.log(name1.toUpperCase() < name2.toUpperCase());  // true ✓
```

Example 2: Password Validation

```javascript
let password = "Abc123";

// Check if password starts with uppercase
if (password[0] >= 'A' && password[0] <= 'Z') {
    console.log("Starts with uppercase");
}

// Check if password contains lowercase
let hasLowercase = false;
for (let char of password) {
    if (char >= 'a' && char <= 'z') {
        hasLowercase = true;
        break;
    }
}
console.log("Has lowercase:", hasLowercase);  // true
```

Example 3: Checking Character Types

```javascript
let char = 'A';

// Check if uppercase letter
if (char >= 'A' && char <= 'Z') {
    console.log("Uppercase letter");
}

// Check if lowercase letter
if (char >= 'a' && char <= 'z') {
    console.log("Lowercase letter");
}

// Check if digit
if (char >= '0' && char <= '9') {
    console.log("Digit");
}
```

ASCII Reference Table

Common ASCII values to remember:

Character	ASCII Value
'0' - '9'	48 - 57
'A' - 'Z'	65 - 90
'a' - 'z'	97 - 122
Space ' '	32

Order: Numbers < Uppercase < Lowercase

```javascript
console.log('9' < 'A');   // true (57 < 65)
console.log('Z' < 'a');   // true (90 < 97)
console.log('0' < 'a');   // true (48 < 97)
```

Case-Insensitive Comparison

To compare strings ignoring case:

```javascript
let str1 = "Apple";
let str2 = "apple";

// Case-sensitive (different)
console.log(str1 === str2);  // false

// Case-insensitive (convert to same case)
console.log(str1.toLowerCase() === str2.toLowerCase());  // true
console.log(str1.toUpperCase() === str2.toUpperCase());  // true

// Case-insensitive comparison function
function equalIgnoreCase(str1, str2) {
    return str1.toLowerCase() === str2.toLowerCase();
}

console.log(equalIgnoreCase("Hello", "HELLO"));  // true
console.log(equalIgnoreCase("World", "WORLD"));  // true
```

Lexicographical (Dictionary) Order

For proper alphabetical sorting:

```javascript
// Incorrect: Direct comparison
let words = ["apple", "Apple", "banana", "Banana"];
words.sort();
console.log(words);  // ["Apple", "Banana", "apple", "banana"]
// Uppercase comes first!

// Correct: Case-insensitive sort
words.sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase()));
console.log(words);  // ["apple", "Apple", "banana", "Banana"]
```

Summary of String Comparison Rules

1. **Character-by-character comparison**: Compares from left to right
2. **First different character decides**: The string with the smaller character is "less"
3. **Uppercase < Lowercase**: All uppercase letters come before lowercase in ASCII
4. **Number strings**: Compared as strings (e.g., "10" < "2" is true)
5. **Case matters**: "Apple" ≠ "apple" (use `.toLowerCase()` for case-insensitive)

Common Pitfalls

```javascript
// ❌ MISTAKE: Assuming alphabetical order
console.log("Zoo" < "apple");  // true (but Z is "after" a alphabetically!)

// ✓ FIX: Convert to same case
console.log("Zoo".toLowerCase() < "apple".toLowerCase());  // false

// ❌ MISTAKE: Comparing number strings
console.log("10" > "9");  // false ("1" < "9" in ASCII)

// ✓ FIX: Convert to numbers
console.log(Number("10") > Number("9"));  // true
```

Key Takeaways

Essential Concepts to Remember

1. **Identifiers** must follow strict rules: start with letter/ , no spaces, no

reserved words

2. **camelCase** is the JavaScript standard for variable and function names
3. **Booleans** represent `true` or `false` - used for logic and conditions
4. **JavaScript is dynamically typed** - variable types can change (unlike TypeScript)
5. **Strings** can use single quotes, double quotes, or backticks
6. **String indexing** starts at 0 - first character is `string[0]`
7. **Concatenation** joins strings with `+` operator
8. **Undefined** = not assigned yet; **Null** = intentionally empty
9. **Template literals** (backticks) make string interpolation easier with `${}`
10. **Comparison operators** return boolean values
11. **Use `===` and `!==`** for strict equality (checks type and value)
12. **String comparison** uses ASCII/Unicode values, not alphabetical order

Common Beginner Mistakes to Avoid

1. **Starting identifiers with numbers**

```javascript
let 1stPlace;  // ✗ Error
let firstPlace;  // ✓ Correct
```

2. **Using spaces in variable names**

```javascript
let user name;  // ✗ Error
let userName;   // ✓ Correct
```

3. **Forgetting string quotes**

```javascript
let name = Alice;   // ✗ Error
let name = "Alice"; // ✓ Correct
```

4. **Confusing `==` with `===`**

```javascript
5 == "5"   // true ⚠️ (type conversion)
5 === "5"  // false ✓ (strict check)
```

5. **Case sensitivity in string comparison**

```javascript
"Apple" === "apple"  // false
"Apple".toLowerCase() === "apple".toLowerCase()  // true
```

6. **Number string comparison**

```javascript
"10" < "2"    // true ⚠️ (string comparison)
10 < 2        // false ✓ (number comparison)
```

Best Practices

1. **Always use descriptive variable names**
   - Bad: `let x = 25;`
   - Good: `let userAge = 25;`
2. **Use `const` for values that don't change**

```javascript
const MAX_USERS = 100;
const API_URL = "https://api.example.com";
```

3. **Prefer template literals over concatenation**

```javascript
// Harder to read
let msg = "Hello " + name + ", you are " + age + " years old.";

// Easier to read
let msg = `Hello ${name}, you are ${age} years old.`;
```

4. **Always use strict equality**

```javascript
if (value === 10) { }    // ✓ Good
if (value == 10) { }     // ⚠ Avoid
```

5. **Be explicit with type conversions**

```javascript
let userInput = "25";
let age = Number(userInput);  // Convert explicitly
```

Practice Exercises

Try these exercises to test your understanding:

Exercise 1: Valid Identifiers

Identify which of these are valid variable names:

```javascript
a) user_name
b) 123abc
c) $price
d) first-name
e) _private
f) let
```

<details> <summary>Click for answers</summary>

- a) ✓ Valid
- b) ✗ Invalid (starts with number)
- c) ✓ Valid
- d) ✗ Invalid (hyphens not allowed)
- e) ✓ Valid
- f) ✗ Invalid (reserved keyword)

</details>

Exercise 2: String Indexing

```javascript
let word = "JavaScript";
// What is:
// word[0]?
// word[4]?
// word[word.length - 1]?
```

<details> <summary>Click for answers</summary> ```javascript word[0] // "J" word[4] // "S" word[word.length - 1] // "t" ``` </details>

Exercise 3: Comparison Results

Predict the output:

```javascript
console.log(10 > 5);
console.log(5 === "5");
console.log('a' > 'A');
console.log("10" < "2");
```

<details> <summary>Click for answers</summary> ```javascript console.log(10 > 5); // true console.log(5 === "5"); // false (different types) console.log('a' > 'A'); // true (97 > 65) console.log("10" < "2"); // true (string comparison: "1" < "2") ``` </details>

Next Steps

In upcoming sessions, we'll cover:

- **Conditional Statements** (if-else, switch)
- **Logical Operators** (&&, ||, !)
- **Loops** (for, while, do-while)
- **Functions**
- **Arrays**
- **Objects**

Keep practicing and happy coding! 🚀

Quick Reference Card

```javascript
```

```
// Valid Identifiers
let userName;       // ✓ camelCase
let user_name;      // ✓ snake_case
let $price;         // ✓ Starts with $
let _private;       // ✓ Starts with _

// Strings
let str1 = "Hello";         // Double quotes
let str2 = 'Hello';         // Single quotes
let str3 = `Hello ${name}`; // Template literal

// String Indexing
let city = "London";
city[0]  // "L" (first character)
city[5]  // "n" (last character)

// Concatenation
"Hello" + " " + "World"  // "Hello World"

// Template Literals
`My name is ${name} and I'm ${age} years old.`

// Comparison Operators
10 > 5      // true
5 < 10      // true
5 >= 5      // true
5 <= 5      // true
5 != 4      // true
5 == "5"    // true (loose)
5 === "5"   // false (strict) ✓ Use this!

// String Comparison
'a' > 'A'            // true (lowercase > uppercase)
'apple' < 'banana'   // true (a < b)
"10" < "2"           // true (string comparison)

// Null vs Undefined
let x;          // undefined
let y = null;   // null
```

Remember: The best way to learn is by doing. Open your browser console and experiment with these concepts. Try breaking things to understand how they work!